

Software Engineering Lab (SE Lab) — Lab 3 Report

Component Modelling & Architectural Pattern Selection

Student Name:	Arjun Srikanth	SRN:	PES1UG24AM920
Class / Section:	5th Sem B.Tech CSE — Section H	Lab Duration:	90 Minutes
Assigned Scenario:	#48: Incident Escalation & On-Call Rotation Engine	Domain:	Developer Tools & IT Operations
Handout Scenario:	Self-Service Coffee Kiosk System (Café Ordering)	Tool Used:	UML 2.5 Component Modelling
GitHub Repository:	github.com/arjunsrikanthh/PES1UG24AM920_se_lab		

1. Comparative Analysis of Architectural Styles

In accordance with Lab 3 requirements, three candidate architectural styles were evaluated against key quality attributes (availability, fault tolerance, peripheral control, and latency):

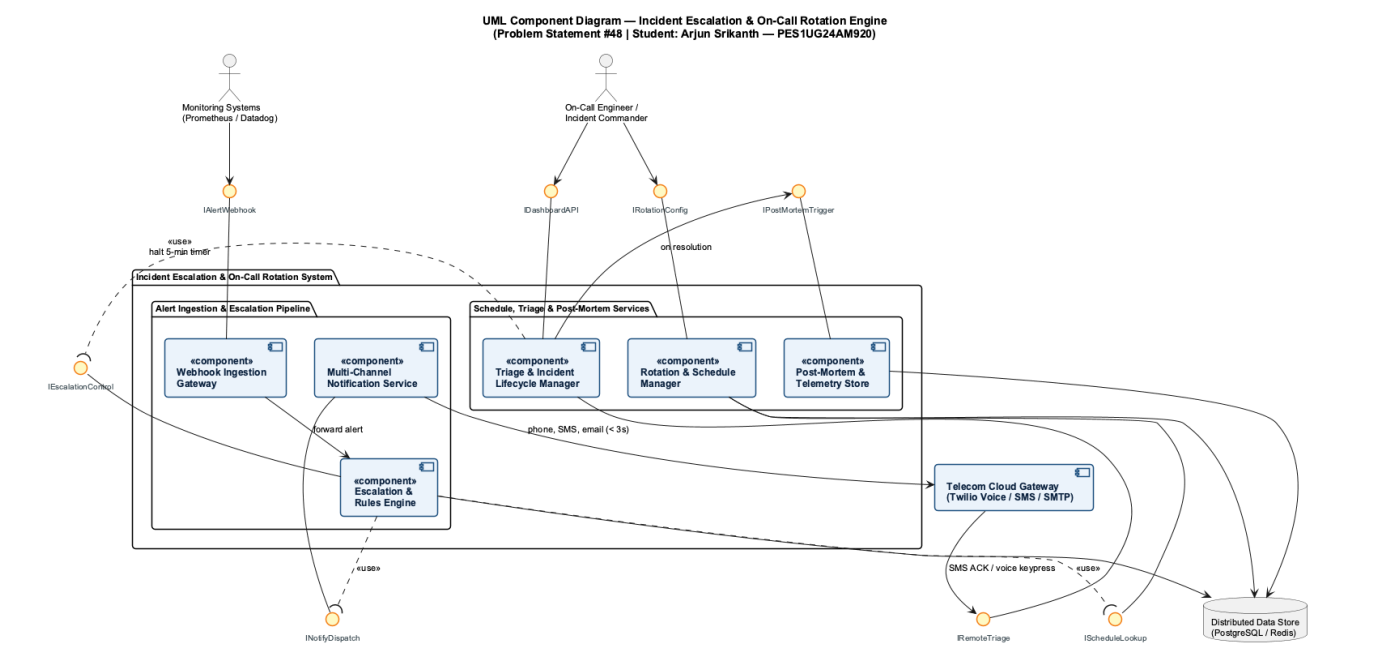
Architectural Style	Structural Organization	Key Advantages (Pros)	Key Limitations (Cons)
Layered (N-Tier)	Horizontal tiers: Presentation, Business Logic, and Data Access with strict downward dependencies.	• Clear separation of concerns • Intuitive mental model & testing • Low operational complexity • Easy component isolation	• Performance sinkhole overhead • Monolithic blast radius • Inflexible uniform scaling • Cascading layer coupling
Microservices Architecture	Autonomous services organized around business domains, interacting via gRPC, REST, and message queues.	• Fault isolation prevents cascades • Targeted horizontal autoscaling • Independent service deployments • Polyglot technology stacks	• High operational & network overhead • Eventual consistency challenges • Complex distributed telemetry • Network partition vulnerability
Client-Server Architecture	Centralized server executing business logic & data access, serving distributed client nodes.	• Centralized state governance • Guaranteed ACID consistency • Lightweight client compute • Straightforward administration	• Central server is a Single Point of Failure • Scalability bottleneck under surge • Network drop halts client workflows • Tight client-server coupling

Synthesis: Microservices Architecture is selected for the distributed incident alerting platform (Problem Statement #48) to guarantee fault isolation and independent burst scaling. Layered Component Architecture is selected for the standalone kiosk (Coffee Kiosk) to provide deterministic sequential execution and direct local hardware peripheral access.

2. Assigned Scenario (Problem Statement #48): Incident Escalation Engine

Architecture Selection: "We chose an Event-Driven Microservices Architecture for the Incident Escalation & On-Call Rotation Engine."

- **Architectural Choice:** Event-Driven Microservices Architecture consisting of 6 autonomous containerized services communicating asynchronously via gRPC and event message streams, backed by PostgreSQL and Redis.
- **Reason 1 (Fault Isolation):** In P1 production incidents, observability systems flood alerts ($\geq 500/\text{sec}$). Decoupling Notification Service prevents telecom carrier rate limits (Twilio Voice/SMS) from stalling the Ingestion Gateway or freezing SLA timers.
- **Reason 2 (Targeted Autoscaling):** Alert ingestion experiences volatile bursts, while rotation schedule administration is low-frequency and read-heavy. Microservices permit autoscaling solely Ingestion and Notification pods to meet the $< 3\text{s}$ dispatch SLA (**NFR-001**).
- **Security Advantage:** Zero-Trust DMZ perimeter enforces HMAC-SHA256 signature verification. Inter-service links use mutual TLS (mTLS). Sensitive engineer contact details are restricted to Notification Service with AES-256 field encryption (**NFR-002**).
- **Performance Benefit:** Ingress webhooks acknowledge within $< 100\text{ms}$. Decoupled asynchronous workers dispatch multi-channel alerts (phone, SMS, email) concurrently within 3 seconds under peak load.

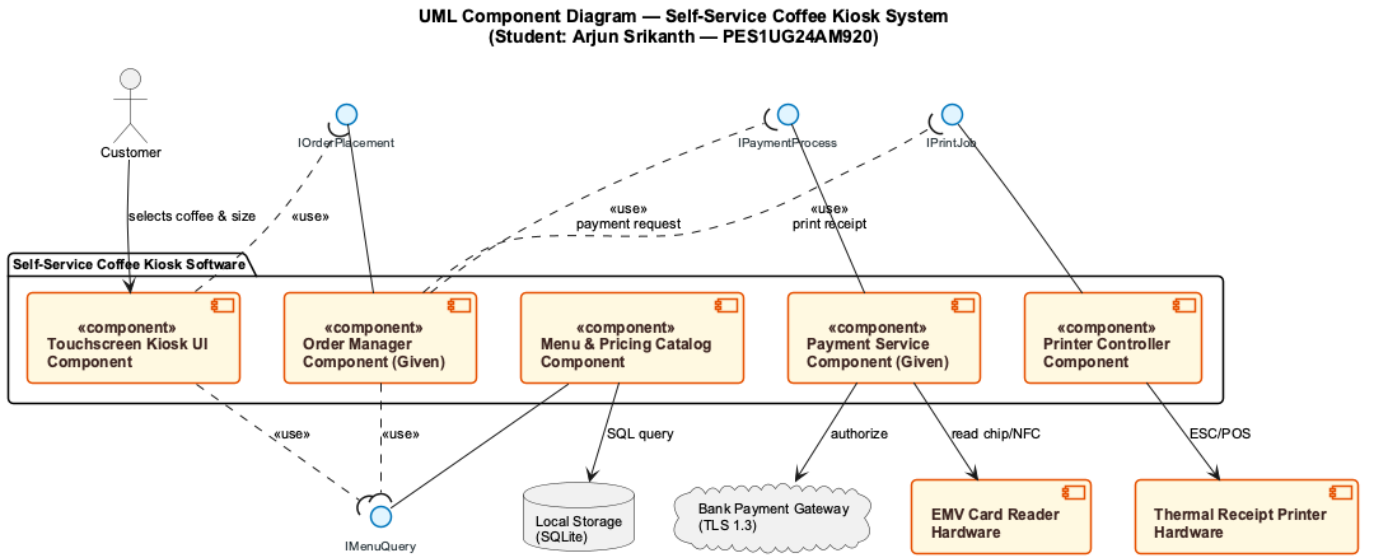


Component Name	Stereotype	Provided Interfaces (Ball)	Required Interfaces (Socket)
Webhook Ingestion Gateway	«component»	IAlertWebhook (HTTPS POST)	IEscalationControl (Event Bus)
Rotation & Schedule Manager	«component»	IScheduleLookup, IRotationConfig	ISqlPersistence (Postgres TLS)
Escalation & Rules Engine	«component»	IEscalationControl (SLA/Timer)	IScheduleLookup, INotifyDispatch
Multi-Channel Notification Svc	«component»	INotifyDispatch (Async Queue)	ITelecomGateway (Twilio API)
Triage & Lifecycle Manager	«component»	IDashboardAPI, IRemoteTriage	IEscalationControl, IPostMortemTrigger
Post-Mortem & Telemetry Store	«component»	IPostMortemTrigger (Internal Event)	ISqlPersistence (Audit Store)

3. Handout Scenario: Self-Service Coffee Kiosk System

Architecture Selection: "We chose a Layered Component-Based Architecture for the Self-Service Coffee Kiosk System."

- **Architectural Choice:** Layered Component-Based Architecture partitioned into Presentation (Touchscreen UI), Business Logic (Order Manager & Menu Catalog), and Hardware Integration (Payment Service & Printer Controller) hosted on kiosk hardware.
- **Reason 1 (Hardware-Software Locality):** The kiosk is an integrated terminal connected to local hardware (touchscreen, EMV card reader, thermal receipt printer). Layered architecture handles low-level USB/Serial IO directly without network dependency.
- **Reason 2 (Deterministic Sequential Flow):** Ordering follows an atomic sequential lifecycle: Select Coffee (Espresso/Latte/Americano) → Select Size (Small/Large) → Authorize Card → Print Receipt. The Order Manager coordinates this flow without distributed transaction overhead.
- **Security Advantage:** PCI-DSS Scope Minimization via Hardware Sandboxing. Payment Service interfaces directly with EMV PIN-pad hardware via encrypted serial IO and talks to bank gateway over TLS 1.3. Card numbers/PINs never enter Order Manager or UI memory space.
- **Performance Benefit:** Zero-Latency Touch UI & Instant Receipts. Menu and pricing data are cached locally for < 16ms UI responsiveness. The Printer Controller directly streams binary ESC/POS bytecode, dispensing receipts in < 1s after payment authorization.



Component Name	Stereotype	Provided Interfaces (Ball)	Required Interfaces (Socket)
Touchscreen Kiosk UI	«component»	—	IOrderPlacement, IMenuQuery
Order Manager Component (Given)	«component»	IOrderPlacement (IPC API)	IMenuQuery, IPaymentProcess, IPrintJob
Payment Service Component (Given)	«component»	IPaymentProcess (Payment API)	ICardReaderDriver, IBankGatewayAPI
Menu & Pricing Catalog	«component»	IMenuQuery (Catalog Lookup)	IDataQuery (Local SQLite DB)
Printer Controller Component	«component»	IPrintJob (Receipt API)	IESCPOSDriver (USB Serial IO)

4. UML 2.5 Standards & Deliverables Verification Checklist

4.1 UML 2.5 Component Modelling Standards Applied:

- **Component Notation:** Rectangular black-box modules with «component» stereotype and component icon symbol.
- **Provided Interface (Ball / Lollipop):** Services offered by a component to other modules (e.g., IOrderPlacement, IScheduleLookup).
- **Required Interface (Socket):** External dependencies needed by a component (e.g., IBankGatewayAPI, INotifyDispatch).
- **Explicit Ports:** Boundaries on components defining distinct interaction contracts with external hardware and networks.
- **Usage Dependencies:** Dashed arrows with «use» indicating behavioral invocations and data flows.

4.2 Handout Deliverables Compliance Verification:

Deliverable / Criterion	Handout Specification	Achieved Status	Verification Details
Component Count	≥ 5 components	PASSED (6 & 5)	6 components (PS #48) and 5 components (Coffee Kiosk)
Interface Count	≥ 4 interfaces	PASSED (8 & 8)	8 ball/socket interfaces modeled in each diagram
Given Components	Order Manager & Payment Svc	PASSED	Both explicitly present in Coffee Kiosk diagram
Given Interface	Order Mgr ↔ Payment Svc	PASSED	IPaymentProcess interface modeled between them
Architectural Evaluation	Layered, Microservices, C/S	PASSED	Structured comparative table with pros and cons (Sec 1)
Written Justification	Choice, 2 Reasons, Sec, Perf	PASSED	All required headings fully documented for both scenarios
Export Formats	PNG / PDF diagrams + Word doc	PASSED	Vector PDF, high-res PNG, DOCX, and PDF report generated
Repository Push	Push to student GitHub repo	PASSED	Synchronized to github.com/arjunsrikanthh/PES1UG24AM920_se_lab

Academic Submission Certification:

I hereby certify that the architectural analysis, UML 2.5 component models, interface definitions, and written justifications presented in this report represent original coursework completed for the Software Engineering Lab (CS304) at PES University, Bengaluru.

Student Signature: Arjun Srikanth (PES1UG24AM920)

Date: September 9, 2026